Perceptrón a viacvrstvový perceptrón

vrstvové dopredné siete (layered NN) - jednovrstvové (single-layer) a viacvrstvové (multilayer)

*úplne prepojené siete (**FCN**, fully connected networks)*

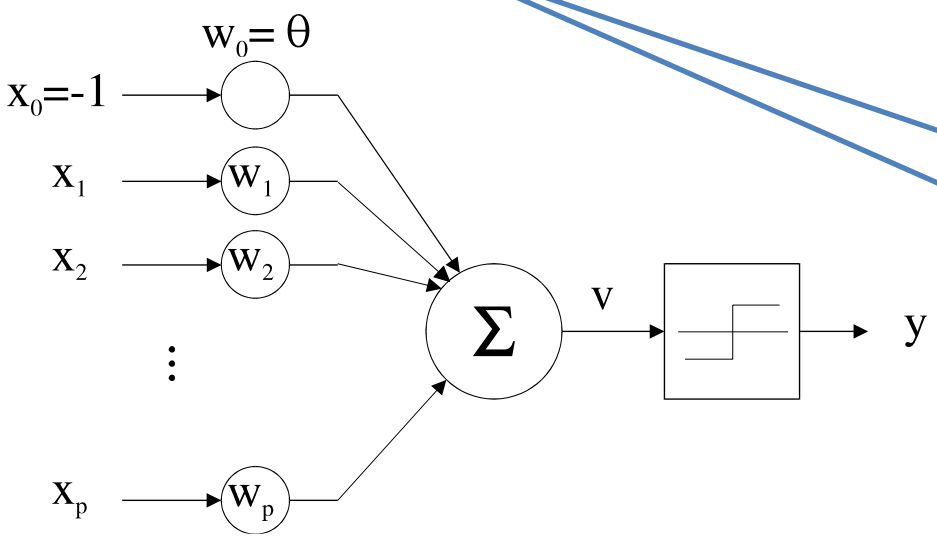
TÉMY

- ***Perceptrón (jednovrstvový)***
  - *algoritmus učenia*
  - *lineárna a nelineárna separovateľnosť*

- ***Viacvrstvový perceptrón (MLP, multilayer perceptron)***
  - *algoritmus spätného šírenia chýb (backpropagation algorithm, error-backpropagation algorithm, BP alg.)*
    - *odvodenie algoritmu pre výstupný neurón*
    - *odvodenie algoritmu pre skrytý neurón*
  - *vysvetlenie spätného šírenia chýb*
  - *sigmoidálna nelinearita*
  - *miznúci a explodujúci gradient*
  - *rýchlosť učenia*
  - *momentová technika*
  - *vzorkový a dávkový mód*
  - *sumarizácia algoritmu spätného šírenia*

  - *prídavok: gradient descent a zanikajúci a explodujúci gradient*

Perceptrón (jednovrstvový), (“single-layer perceptron”)



jednoduchá neurónová sieť pre klasifikáciu špeciálneho typu vzoriek, ktoré sú *lineárne separovateľné*

$$y(n) = \mathrm{sgn}\left[\mathbf{w}^T(n)\mathbf{x}(n)\right]$$

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \eta\left[d(n) - y(n)\right]\mathbf{x}(n)$$

$$d(n) = \begin{cases} +1 & \text{ak } \mathbf{x}(n) \in C_1 \\ -1 & \text{ak } \mathbf{x}(n) \in C_2 \end{cases}$$

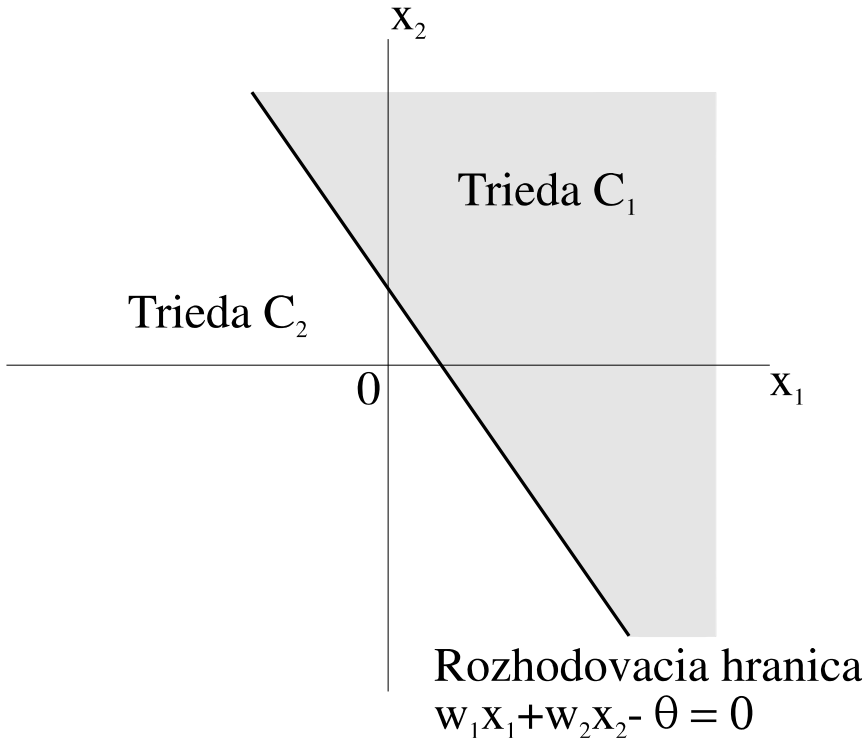
klasické učenie korigujúce chybu, kde η je parameter rýchlosti učenia, a rozdiel $d(n) - y(n)$ reprezentuje chybový signál

- <u>lineárna a nelineárna separovateľnosť</u>

dva rozhodovacie regióny oddelené hyperrovinou definovanou rovnicou

$$\sum_{i=1}^{p} w_i x_i - \theta = 0$$

nadrovina
(hyperplane)



Lineárna separovateľnosť - dvojrozmerné vstupy, dve triedy

Učenie Perceptrónu

- iterácia n: $(p+1)$-rozmerné vektory vstupov $\boldsymbol{x}(n)$ a váh $\boldsymbol{w}(n)$ $\boldsymbol{x}(n) = \left[-1, x_1(n), x_2(n), \ldots, x_p(n)\right]^T$, $\boldsymbol{w}(n) = \left[\theta(n), w_1(n), w_2(n), \ldots, w_p(n)\right]^T$, prah $\theta(n)$, aktuálna odpoveď $y(n)$, požadovaná odpoveď $d(n)$, a parameter rýchlosti učenia η $(0 < \eta \leq 1)$

  - *Krok 1 - inicializácia:* Nastav $\boldsymbol{w}(0) = \boldsymbol{0}$. Potom pre čas $n = 1,2,\ldots$ rob nasledovné kroky.
  - *Krok 2 - aktivácia:* V čase n priveď vstupný vektor $\boldsymbol{x}(n)$ a požadovanú odpoveď $d(n)$.
  - *Krok 3 - výpočet aktuálnej odpovede:*

    $$y(n) = sgn[\boldsymbol{w}^T(n)\boldsymbol{x}(n)],$$ sgn je funkcia signum.
  - *Krok 4 - adaptácia váhového vektora:*

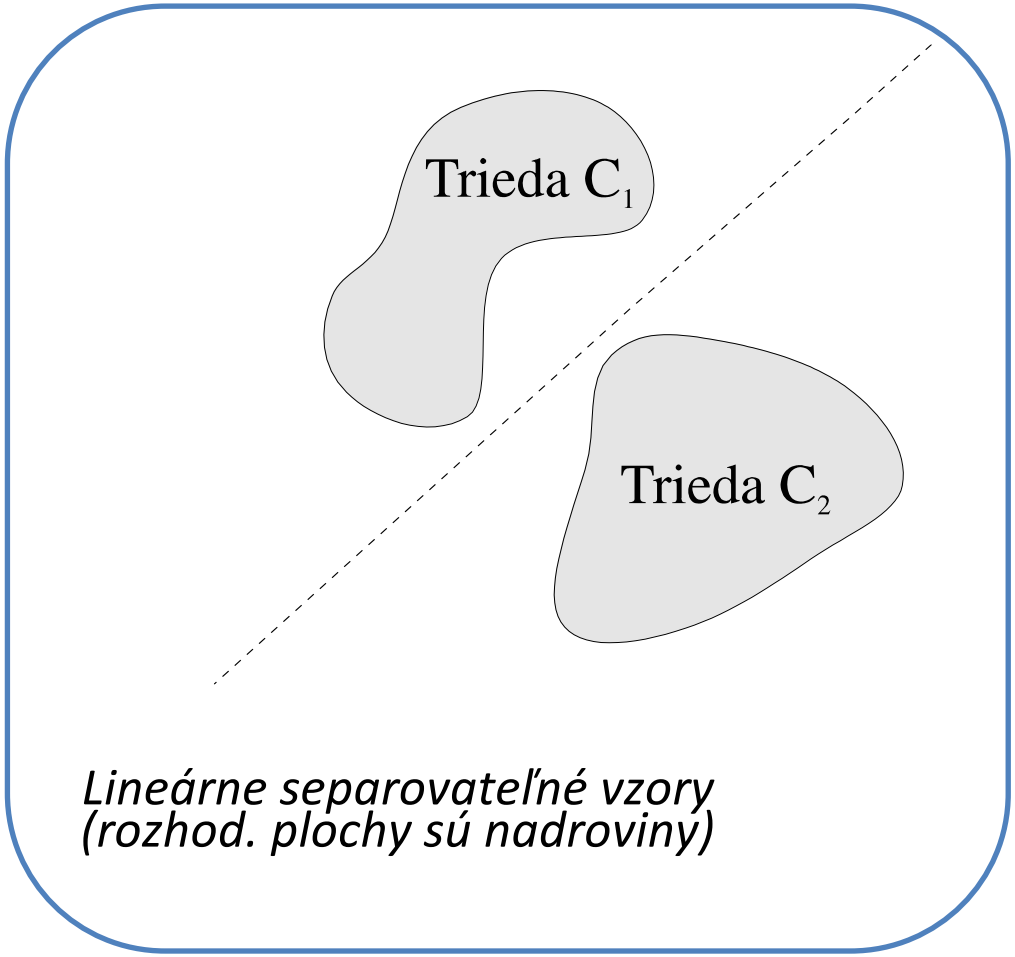
    $$\boldsymbol{w}(n+1) = \boldsymbol{w}(n) + \eta[d(n) - y(n)]\boldsymbol{x}(n)$$

    kde

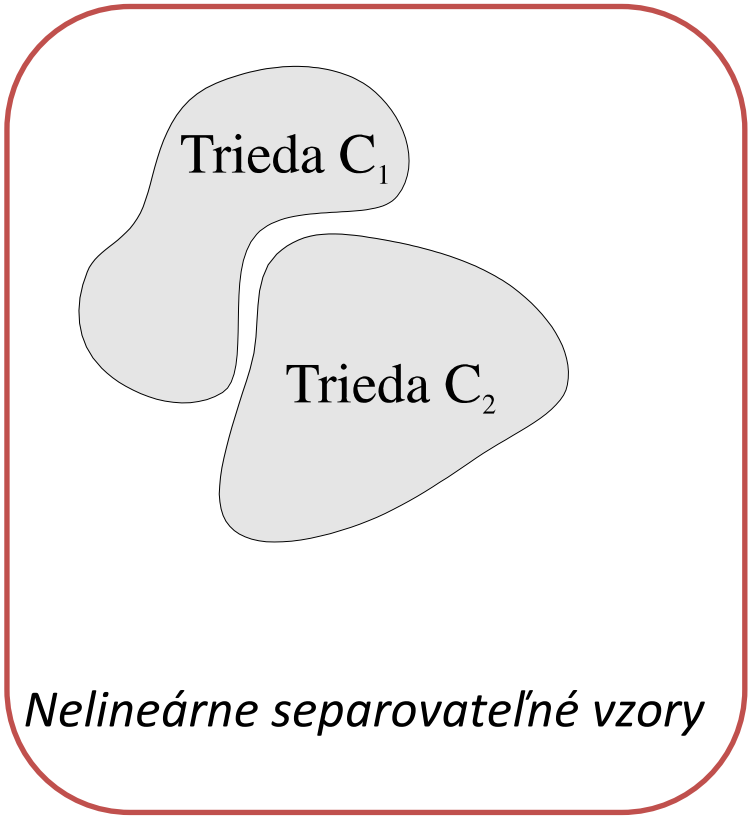
    $$d(n) = \begin{cases} +1 & \text{ak } \mathbf{x}(n) \in C_1 \\ -1 & \text{ak } \mathbf{x}(n) \in C_2 \end{cases}$$
  - *Krok 5:* Inkrementuj n a choď na krok 2.

klasické učenie korigujúce chybu, kde η je parameter rýchlosti učenia, a rozdiel $d(n) - y(n)$ reprezentuje chybový signál

- ešte lineárna a nelineárna separovateľnosť

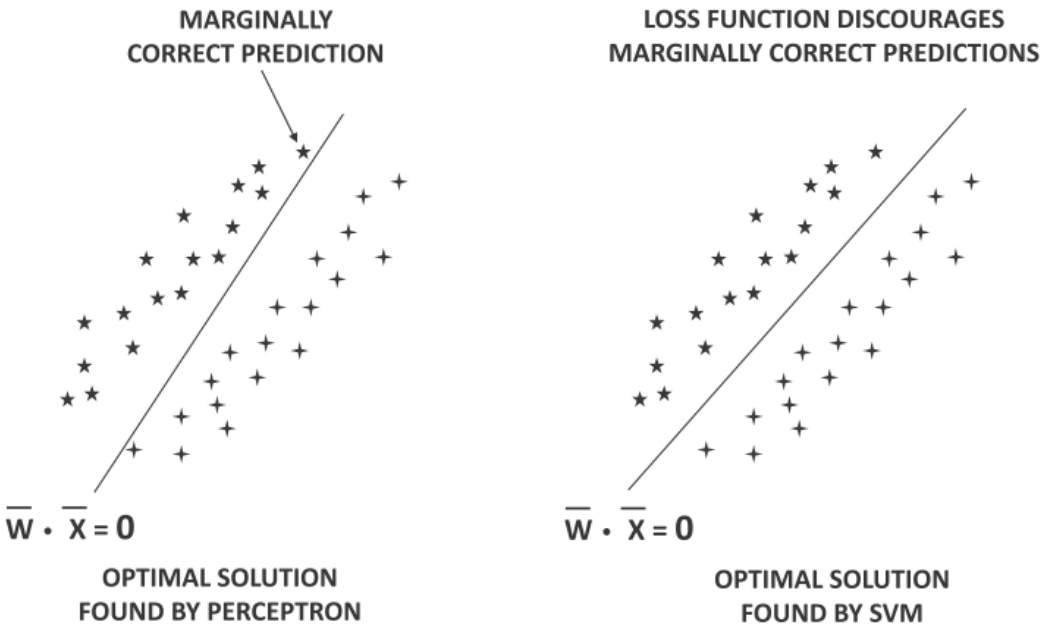


Lineárne separovateľné vzory (rozhod. plochy sú nadroviny)



Nelineárne separovateľné vzory

Perceptron vs Linear SVMs



- The more rigorous condition for the update in a linear SVM ensures that points near the decision boundary *generalize* better to the test data.

Aggarwall,C.,C.: Neural Networks and Deep Learning: A Textbook (Second Edition),Springer, 2018

- možnosti použitia perceptrónu analyzovali Minsky a Papert 1969
- perceptrón nie je schopný (okrem iných logických funkcií) implementovať ani funkciu XOR (exclusive or)

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

- výskum v oblasti neurónových sietí je ukončený na mŕtvom bode
- pre neurónové siete nasledovali “tiché roky” do polovice osemdesiatych rokov

- 1986: PDP group, *viacvrstvový perceptrón* trénovaný algoritmom spätného šírenia

8. október 2024:

Nobelova cena za fyziku: John Hopfield, Geoffrey Hinton,

https://www.theguardian.com/science/2024/oct/08/nobel-prize-physics-john-hopfield-geoffrey-hinton-machine-learning



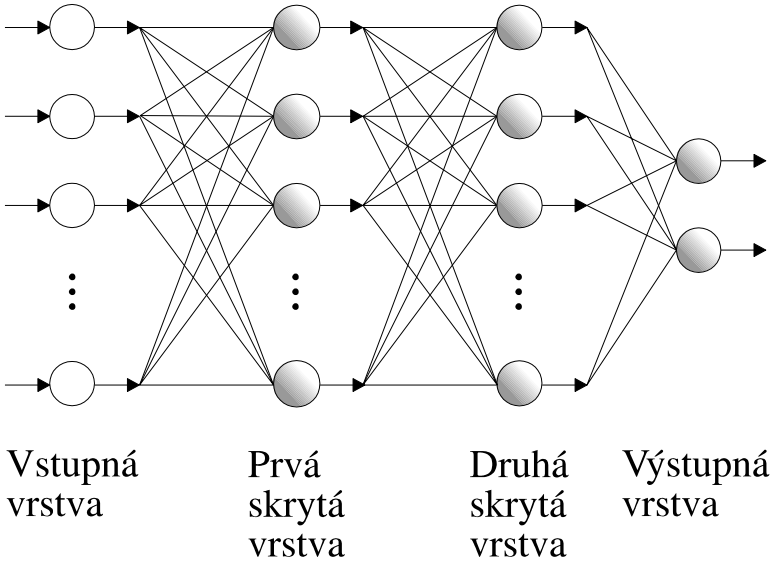
- 1986: PDP group, *viacvrstvový perceptrón* trénovaný algoritmom spätného šírenia:

Rumelhart, David E.; Hinton, Geoffrey E.; Williams, Ronald J. (9 October 1986). "Learning representations by back-propagating errors". Nature. 323 (6088), 533-536, doi:10.1038/323533a0. ISSN 1476-4687. S2CID 205001834.

Viacvrstvový perceptrón (MLP, multilayer perceptron)

- **niekedy aj sieť so spätným šírením (backpropagation network, BP network)**
- **dopredná sieť (viacvrstvová) trénovaná algoritmom spätného šírenia (chýb)**
- **algoritmus spätného šírenia chýb (backpropagation algorithm, error-backpropagation algorithm, BP alg.)**

- jedna alebo viacero *skrytých vrstiev* výpočtových prvkov

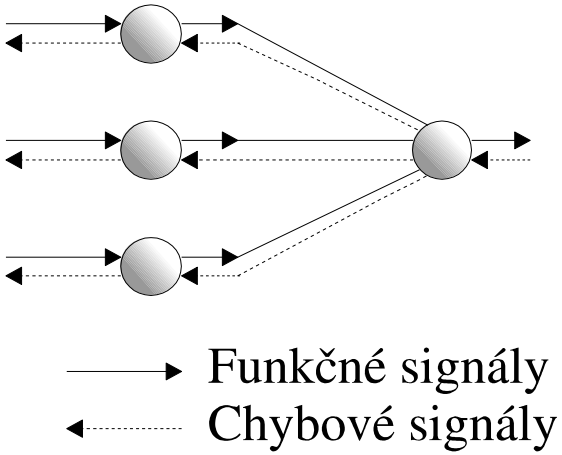


Viacvrstvový perceptrón s dvomi skrytými vrstvami (úplne prepojená sieť)

- hladká nelinearita pre každý neurón (t.j. všade diferencovateľná)
- najčastejšie sigmoidálna nelinearita - napr. logistická funkcia

$$y_j = \frac{1}{1 + exp(-v_j)}$$

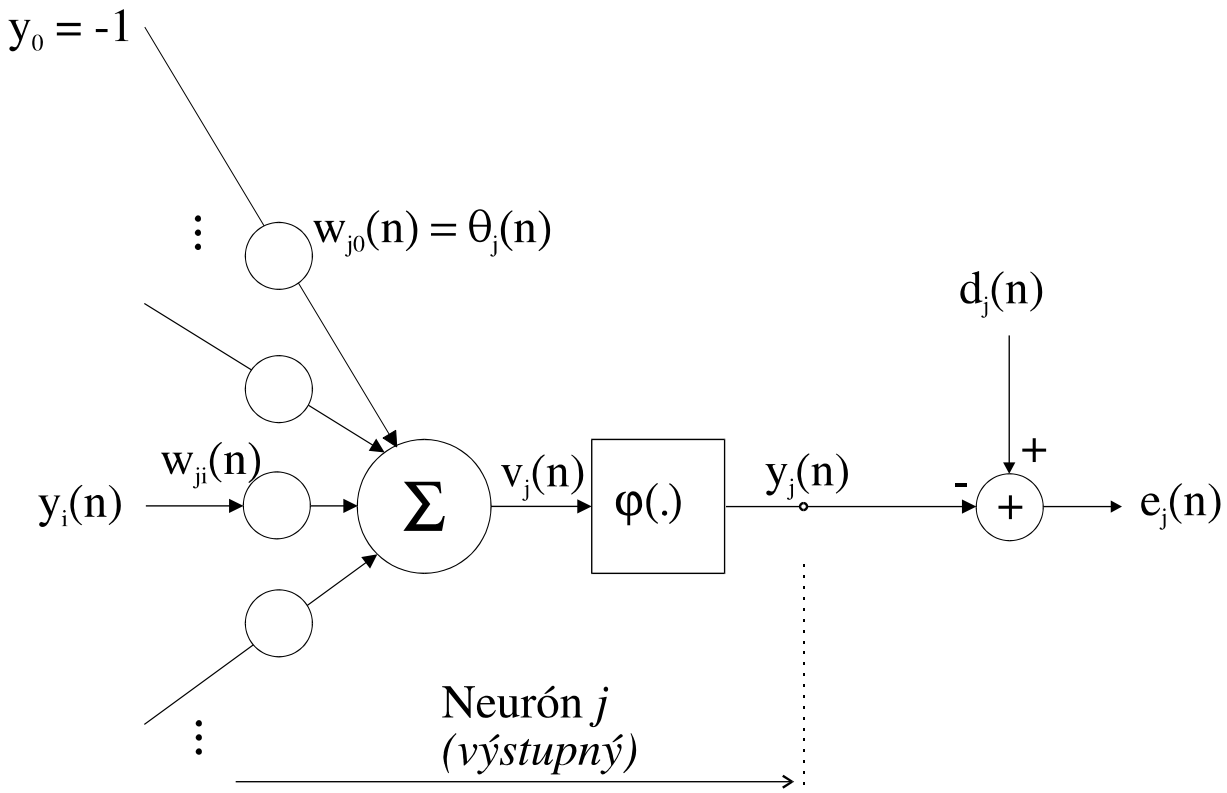
- funkčné a chybové signály:



  - Funkčný signál je signál, ktorý prichádza zo vstupu siete, šíri sa dopredne neurón po neuróne až sa objaví na výstupe siete ako výstupný signál.
  - Chybový signál vzniká vo výstupných neurónoch siete a šíri sa spätne vrstvu po vrstve po sieti (iba počas trénovania).

Algoritmus spätného šírenia (chýb) – (error) backpropagation algorithm

Prípad I: Neurón j je výstupným neurónom



- chybový signál na výstupe neurónu $_j$: $e_j(n) = d_j(n) - y_j(n)$

- suma okamžitých kvadratických chýb: $\varepsilon(n) = \frac{1}{2}\sum_{j \in C} e_j^2\,(n)$

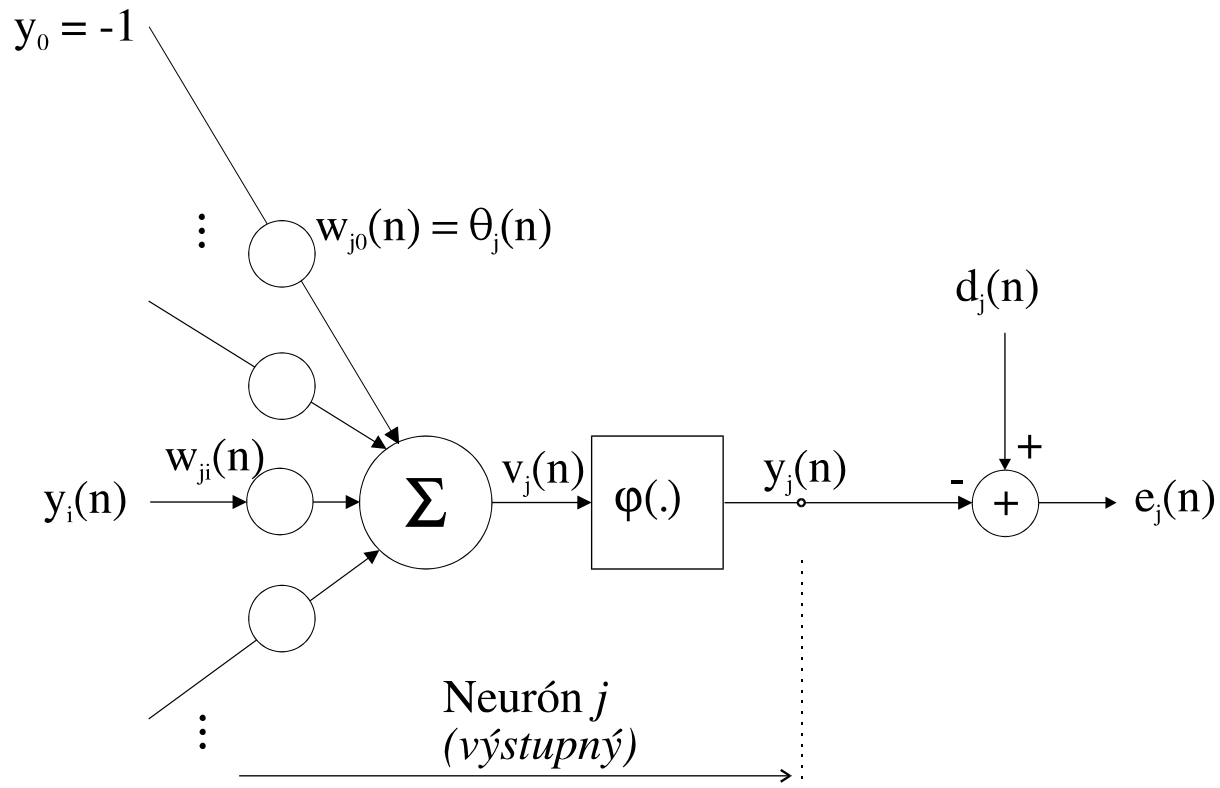
C – množina výstupných neurónov

- priemerná kvadratická chyba: $\varepsilon_{av} = \frac{1}{N}\sum_{n=1}^{N} \varepsilon(n)$

N – počet vzoriek (príkladov) trénovacej množiny

  - $\varepsilon(n)$ alebo ε_{av} - kriteriálna funkcia – minimalizujeme vzhľadom na váhy

- minimalizujme $\varepsilon(n)$ - úpravy váh sa robia na základe chýb počítaných pre každú vzorku privedenú do siete



$$v_j(n) = \sum_{i=0}^{p} w_{ji}(n) y_i(n)$$

$$y_j(n) = \varphi_j\left(v_j(n)\right)$$

- úprava $\Delta w_{ji}(n)$: úmerná okamžitému gradientu $\partial \varepsilon(n) / \partial\, w_{ji}(n)$

$$\frac{\partial \varepsilon(n)}{\partial w_{ji}(n)} = \frac{\partial \varepsilon(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \frac{\partial v_j(n)}{\partial w_{ji}(n)}$$

reťazové pravidlo ☺
(pravidlo pre výpočet parciálnej derivácie zloženej funkcie)

$$\frac{\partial \varepsilon(n)}{\partial e_j(n)} = e_j(n)$$

$$\frac{\partial e_j(n)}{\partial y_j(n)} = -1$$

$$\frac{\partial y_j(n)}{\partial v_j(n)} = \varphi_j'\left(v_j(n)\right)$$

$$\frac{\partial v_j(n)}{\partial w_{ji}(n)} = y_i(n)$$

$$\frac{\partial \varepsilon(n)}{\partial w_{ji}(n)} = -e_j(n)\varphi_j'\left(v_j(n)\right) y_i(n)$$

tzv. delta pravidlo: $$\Delta w_{ji}(n) = -\eta \frac{\partial \varepsilon(n)}{\partial w_{ji}(n)}$$

znamienko mínus znamená gradientový zostup po chybovom povrchu

$$\Delta w_{ji}(n) = \eta \delta_j(n) y_i(n)$$

lokálny gradient $$\delta_j(n) = -\frac{\partial \varepsilon(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} = e_j(n)\varphi_j'\left(v_j(n)\right)$$

$$\Delta w_{ji}(n) = \eta \delta_j(n) y_i(n)$$

$$\begin{pmatrix} \text{úprava} \\ \text{váhy} \\ \Delta w_{ji}(n) \end{pmatrix} = \begin{pmatrix} \text{parameter} \\ \text{rýchlosti učenia} \\ \eta \end{pmatrix} \cdot \begin{pmatrix} \text{lokálny} \\ \text{gradient} \\ \delta_j(n) \end{pmatrix} \cdot \begin{pmatrix} \text{vstupný signál} \\ \text{neurónu j} \\ y_i(n) \end{pmatrix}$$

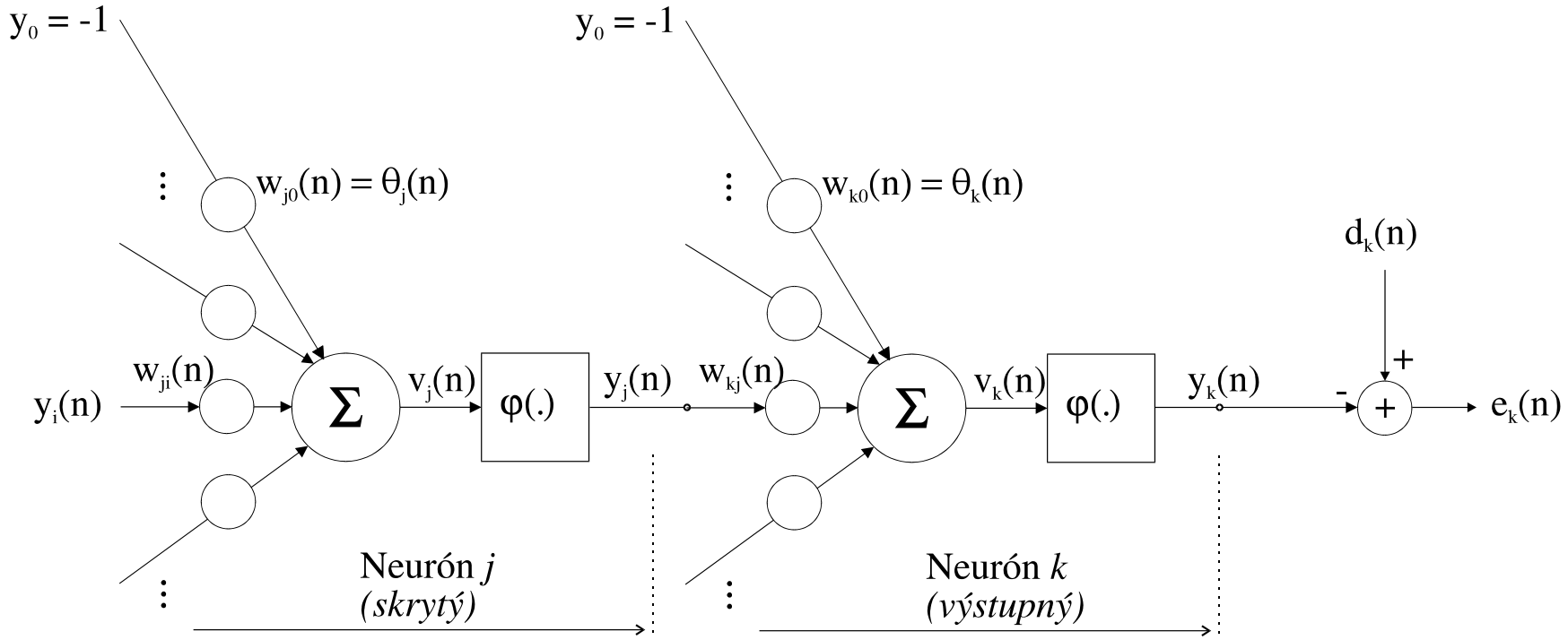
- neurón *j*:
  1. je výstupným neurónom - požadovaná odpoveď je pre výstupné neuróny priamo dostupná

$$\delta_j(n) = e_j(n)\varphi_j'\left(v_j(n)\right)$$

  2. je skrytým neurónom - požadovaná odpoveď je nedostupná

PROBLÉM! – ale odpoveď dáva algoritmus spätného šírenia:
Chybový signál pre skrytý neurón musí byť vypočítaný rekurzívne podľa chybových signálov neurónov, s ktorými je priamo spojený

Prípad II: Neurón j je skrytým neurónom



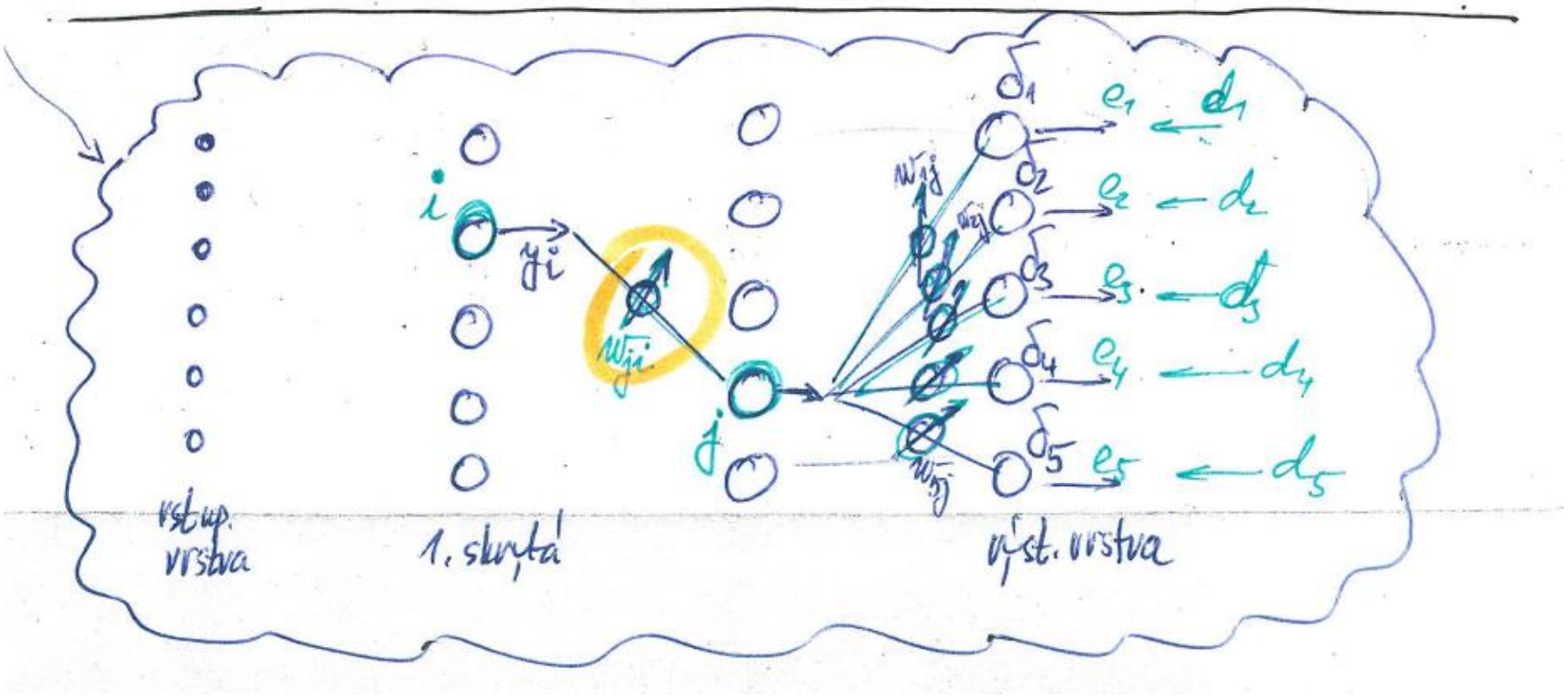
- lokálny gradient pre skrytý neurón:

$$\delta_j(n) = -\frac{\partial \varepsilon(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} = -\frac{\partial \varepsilon(n)}{\partial y_j(n)} \varphi_j'\left(v_j(n)\right)$$

- opäť reťazové pravidlo: $\frac{\partial \varepsilon(n)}{\partial y_j(n)} = \sum_k e_k(n) \frac{\partial e_k(n)}{\partial v_k(n)} \frac{\partial v_k(n)}{\partial y_j(n)}$
- ... (ďalšie úpravy)

$$\delta_j(n) = \varphi_j'\left(v_j(n)\right)\sum_k \delta_k(n) w_{kj}(n)$$

- vysvetlenie spätného šírenia chýb:



$$\delta_j(n) = \varphi_j'\left(v_j(n)\right)\left[w_{1j}(n)\delta_1(n) + w_{2j}(n)\delta_2(n) + \ldots + w_{5j}(n)\delta_5(n)\right]$$

Sumarizácia:

$$w_{ji}(n+1) = w_{ji}(n) + \Delta w_{ji}(n)$$

$$\Delta w_{ji}(n) = \eta \delta_j(n) y_i(n)$$

$$\begin{pmatrix} \text{úprava} \\ \text{váhy} \\ \Delta w_{ji}(n) \end{pmatrix} = \begin{pmatrix} \text{parameter} \\ \text{rýchlosti učenia} \\ \eta \end{pmatrix} \cdot \begin{pmatrix} \text{lokálny} \\ \text{gradient} \\ \delta_j(n) \end{pmatrix} \cdot \begin{pmatrix} \text{vstupný signál} \\ \text{neurónu j} \\ y_i(n) \end{pmatrix}$$

- neurón *j*:
  1. je výstupným neurónom - požadovaná odpoveď je pre výstupné neuróny priamo dostupná

$$\delta_j(n) = e_j(n)\varphi_j'\left(v_j(n)\right)$$

  2. je skrytým neurónom - požadovaná odpoveď je nedostupná

$$\delta_j(n) = \varphi_j'\left(v_j(n)\right)\sum_k \delta_k(n) w_{kj}(n)$$

váhovaná suma lokálnych gradientov pre neuróny v ďalšej skrytej alebo výstupnej vrstve, ktoré sú spojené s neurónom *j*

- **dve fázy výpočtu**

1) dopredná fáza

- synaptické váhy sa nemenia
- funkčné signály siete sa počítajú postupne neurón po neuróne

p je celkový počet vstupov neurónu *j*

$$v_j(n) = \sum_{i=0}^{p} w_{ji}(n) y_i(n) \qquad y_j(n) = \varphi\left(v_j(n)\right)$$

- neurón – vstupný:

$$y_i(n) = x_i(n)$$

- neurón – výstupný:

$$y_j(n) = o_j(n)$$

o znamená output

- dopredná fáza výpočtu začína v prvej skrytej vrstve privedením vstupného vektora a končí sa vo výstupnej vrstve určením chybových signálov pre všetky výstupné neuróny

2) spätná fáza

- začína vo výstupnej vrstve siete, z nej sa chybové signály šíria spätne postupne vrstvu po vrstve a rekurzívne sa počítajú lokálne gradienty δ pre každý neurón

- upravujú sa váhy

$$\Delta w_{ji}(n) = \eta \delta_j(n) y_i(n)$$

- **sigmoidálna nelinearita**

  - výpočet lokálnych gradientov δ - treba vedieť deriváciu aktivačnej funkcie
  - jediná podmienka pre aktivačnú funkciu – *diferencovateľnosť*
  - napr. logistická funkcia:

$$y_j(n) = \varphi_j\left(v_j(n)\right) = \frac{1}{1 + exp\left(-v_j(n)\right)}$$

$$\frac{\partial y_j(n)}{\partial v_j(n)} = \varphi_j^{'}\left(v_j(n)\right) = \frac{exp\left(-v_j(n)\right)}{\left[1 + exp\left(-v_j(n)\right)\right]^2}$$

$$\varphi_j^{'}\left(v_j(n)\right) = y_j(n)\left[1 - y_j(n)\right]$$

- neurón – výstupný:

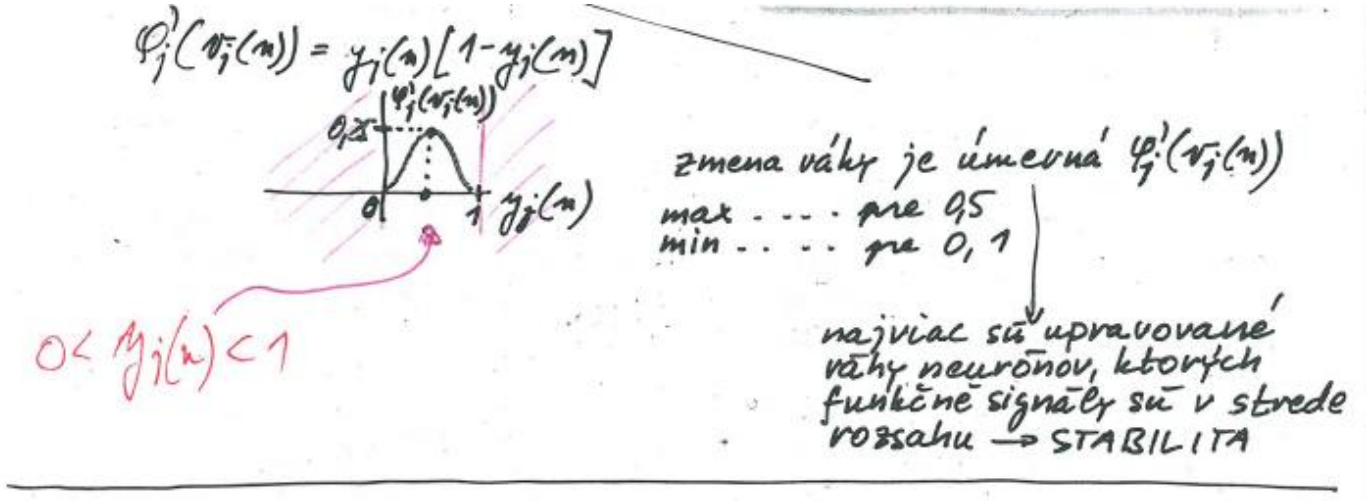
$$y_j(n) = o_j(n)$$

$$\delta_j(n) = e_j(n)\varphi_j'\left(v_j(n)\right)$$
$$= \left[d_j(n) - o_j(n)\right]o_j(n)\left[1 - o_j(n)\right]$$

- neurón – skrytý:

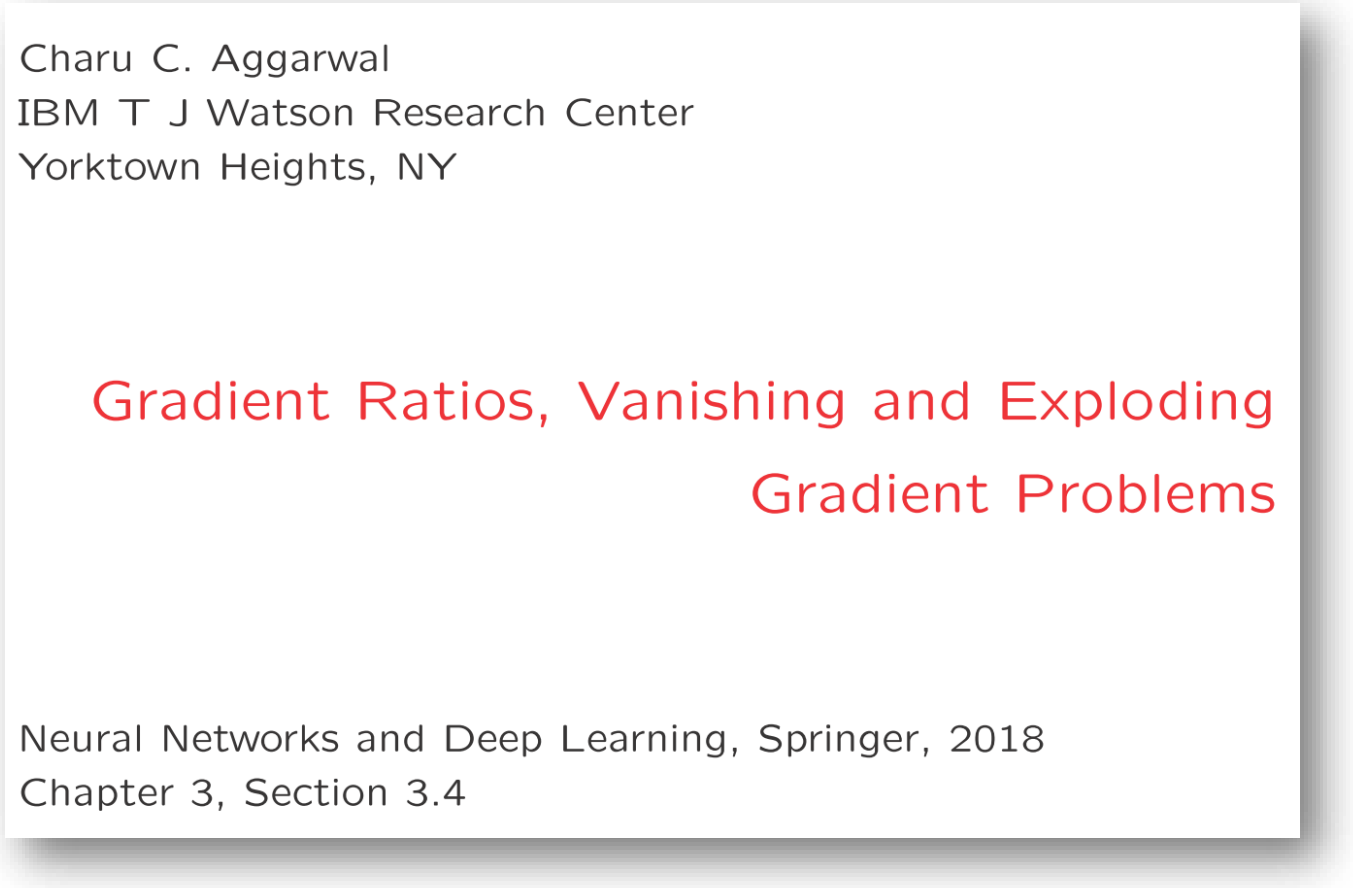
$$\delta_j(n) = \varphi_j'\left(v_j(n)\right)\sum_k \delta_k(n)\, w_{kj}(n)$$
$$= y_j(n)\left[1 - y_j(n)\right]\sum_k \delta_k(n)\, w_{kj}(n)$$

- najviac sú upravované váhy tých neurónov so sigmoidálnou nelinearitou, ktorých výstupné hodnoty sú v strede svojho rozsahu
- toto prispieva k stabilite učenia



Pozri:

[04-MLP pridavok1-Gradient Ratios, Vanishing and Exploding Gradient Problems.pdf]



Aggarwall,C.,C.: Neural Networks and Deep Learning: A Textbook (Second Edition),Springer, 2018

- **rýchlosť učenia**

  - parameter rýchlosti učenia η:
    - veľmi malá hodnota - malé zmeny váh siete, hladká trajektória vo váhovom priestore, učenie siete veľmi pomalé
    - príliš veľká hodnota – veľké zmeny váh, môže to viesť k nestabilite

- **momentová technika algoritmu spätného šírenia**

$$\Delta w_{ji}(n) = \eta \delta_j(n) y_i(n) + \alpha \Delta w_{ji}(n-1)$$

  - α - zvyčajne kladné číslo - *momentová konštanta*
    - pre konvergenciu učenia má platiť

$$0 \leq |\alpha| < 1$$

  - metóda pre zvýšenie rýchlosti učenia
  - predídenie nestability učenia

  - MOŽNOSŤ zabrániť procesu učenia skončiť v lokálnom minime

- **vzorkový a dávkový mód učenia**

SGD -stochastic gradient descent

  - vzorkový mód učenia - váhy upravované po každej prezentácii vzorky (to bolo pôvodne)
    - radšej privedenie niekoľkých vzoriek v každom kroku - **minibatch**

  - dávkový mód učenia - váhy sa upravujú až po privedení všetkých vzoriek trénovacej množiny (t.j. po každej epoche)

gradientový zostup - štandardný, dávkový (batch)

epocha (cyklus) je privedenie úplnej trénovacej množiny

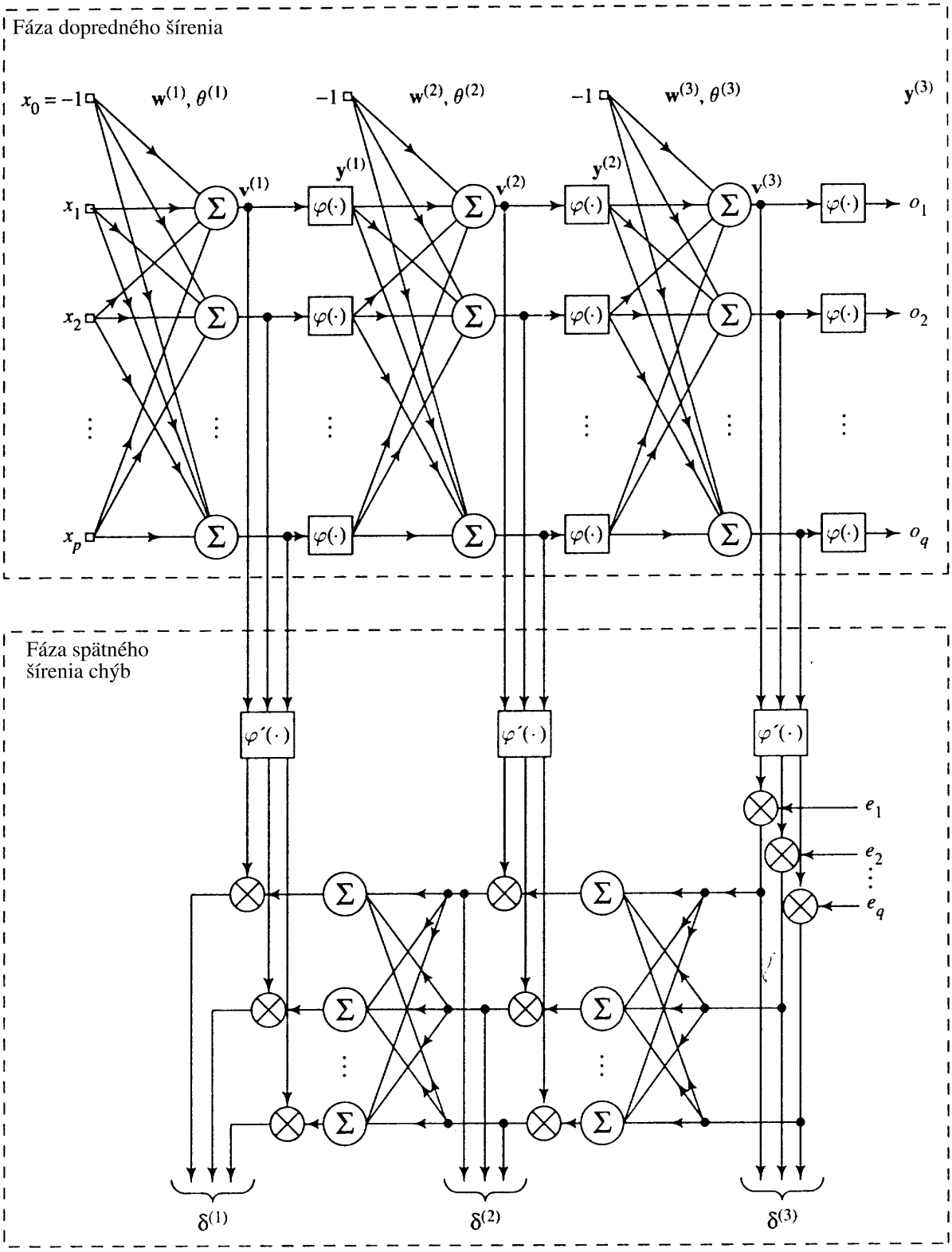
  - najčastejšie sa využíva vzorkový mód činnosti:
    - menej pamäte pre každú váhu
    - možnosť sieti prezentovať vzorky v náhodnom poradí - môže zamedziť ukončeniu učenia v lokálnom minime

  - dávkový mód činnosti umožňuje presne určiť gradientový vektor

  - výber módu závisí od konkrétnej úlohy

Zhrnutie algoritmu spätného šírenia

Učenie viacvrstvového perceptrónu algoritmom spätného šírenia je ilustrované na obrázku [Haykin,1994]. Zahŕňa doprednú fázu výpočtu a aj fázu spätného šírenia chýb.



Označenie v tomto obrázku je nasledovné:

- $\mathbf{w}^{(l)}$ váhový vektor neurónu vo vrstve l
- $\theta^{(l)}$ prah neurónu vo vrstve l
- $\mathbf{v}^{(l)}$ vektor vnútorných aktivít neurónov vo vrstve l
- $\mathbf{y}^{(l)}$ vektor funkčných signálov neurónov vo vrstve l
- l index vrstvy, $l = 0 \ldots L$, $l = 0$ pre vstupnú a $l = L$ pre výstupnú vrstvu
- $?^{(l)}$ vektor lokálnych gradientov neurónov vo vrstve l
- $\mathbf{e}$ chybový vektor, jeho prvky sú $e_1, e_2, \ldots, e_q$

Uvažujme vzorkový mód činnosti. Pre trénovacie údaje $\{[\mathbf{x}(n), \mathbf{d}(n)], n = 1,2,\ldots,N\}$ algoritmus prechádza nasledovnými cyklami:

Krok 1 - inicializácia: Nastav váhy a prahy siete na malé náhodné čísla.

Krok 2 - prezentácia trénovacích údajov: Priveď do siete trénovacie príklady, ktoré tvoria jednu epochu. Pre každý príklad rob postupnosť dopredných a spätných výpočtov podľa bodov 3 a 4.

Krok 3 - dopredný výpočet: Dodaj sieti trénovací príklad $[\mathbf{x}(n), \mathbf{d}(n)]$ Počítaj funkčné signály siete pre jednotlivé neuróny:

$$v_j^{(l)}(n) = \sum_{i=0}^{p} w_{ji}^{(l)}(n) y_i^{(l-1)}(n)$$

kde pre $i = 0$ je $y_0^{(l-1)}(n) = -1$ a $w_{j0}^{(l)}(n) = \theta_j^{(l)}(n)$. Ak je sigmoidálnou nelinearitou logistická funkcia, potom

$$y_j^{(l)}(n) = \frac{1}{1 + \exp\left(-v_j^{(l)}(n)\right)}$$

Ak $l = 0$, potom nastav $y_j^{(0)}(n) = x_j(n)$, ak $l = L$, potom nastav $y_j^{(L)}(n) = o_j(n)$.

Potom počítaj chybový signál

$$e_j(n) = d_j(n) - o_j(n)$$

Krok 4 - spätná fáza výpočtu: Počítaj lokálne gradienty δ siete, postupne od výstupnej smerom k vstupnej vrstve:

$$\delta_j^{(L)}(n) = e_j^{(L)}(n) o_j(n)[1 - o_j(n)],$$

pre neurón j vo výstupnej vrstve L

$$\delta_j^{(l)}(n) = y_j^{(l)}(n)\left[1 - y_j^{(l)}(n)\right] \sum_k \delta_k^{(l+1)}(n) w_{kj}^{(l+1)}(n),$$

pre neurón j v skrytej vrstve l

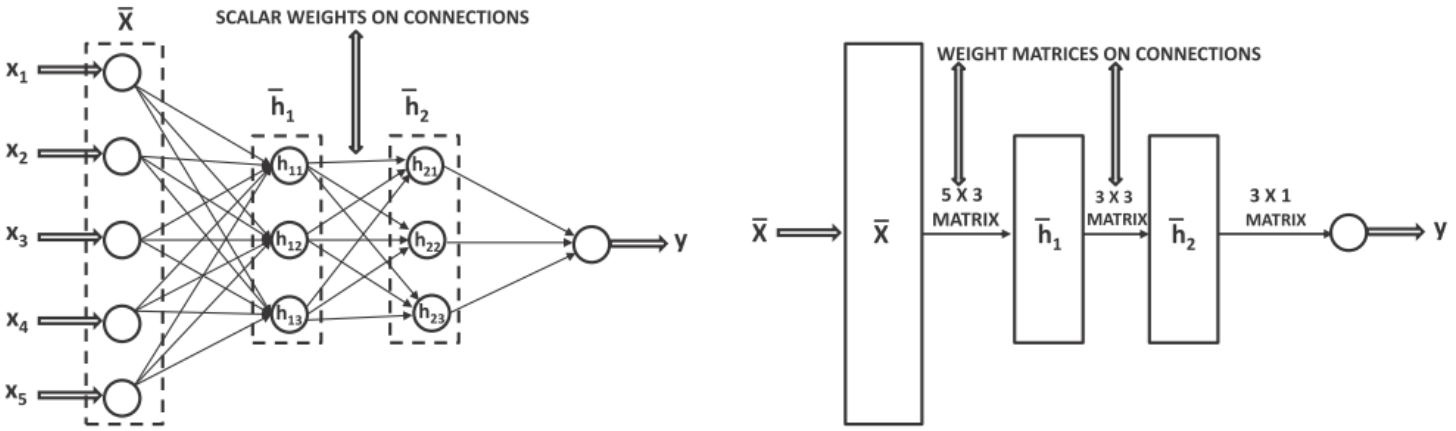
Uprav váhy vo vrstve l:

$$w_{ji}^{(l)}(n+1) = w_{ji}^{(l)}(n) + \alpha\left[w_{ji}^{(l)}(n) - w_{ji}^{(l)}(n-1)\right] + \eta \delta_j^{(l)}(n) y_i^{(l-1)}(n)$$

kde η je parameter rýchlosti učenia a α je momentová konštanta.

Krok 5 - iterovanie: Iteruj výpočet prezentovaním nových epoch trénovacích príkladov, pokiaľ sa váhy siete nestabilizujú a priemerná kvadratická chyba ε_{av} je v minime alebo v akceptovateľne malej hodnote. Poradie prezentácie trénovacích príkladov by sa malo pre rôzne epochy náhodne meniť. Momentová konštanta a parameter rýchlosti učenia sa s rastúcim počtom iterácií upravujú (zvyčajne znižujú).

Scalar versus Vector Diagrams



- Hidden vectors $\overline{h}_1 \ldots \overline{h}_k$ and weight matrices $W_1 \ldots W_{k+1}$.

$$\overline{h}_1 = \Phi(W_1^T \overline{x}) \quad \text{[Input to Hidden Layer]}$$
$$\overline{h}_{p+1} = \Phi(W_{p+1}^T \overline{h}_p) \quad \forall p \in \{1 \ldots k-1\} \quad \text{[Hidden to Hidden Layer]}$$
$$\overline{o} = \Phi(W_{k+1}^T \overline{h}_k) \quad \text{[Hidden to Output Layer]}$$

- $\Phi(\cdot)$ is typically applied element-wise (other than softmax).

Aggarwall,C.,C.: Neural Networks and Deep Learning: A Textbook (Second Edition),Springer, 2018